Chapter 14: Augmenting Data Structures

1) There are some engineering situations that do not require more than a "textbook" data structure - such as a doubly linked list, a hash table or a binary search tree - but some others require a dash of creativity. Only in rare situations will you need to create an entirely new kind of data structure. More often, it will suffice to augment a textbook data structure by storing additional information in it. You can then program new operations for the data structure to support the desired application.

2) Dynamic order statistics

→ the ith order statistic of a set of n elements, where $i \in \{1, 2, \ldots, n\}$ is that element of the set with the ith smallest key.

→ In general any order statistic could be retrieved in $O(n)$ time from an unordered set. Red-black trees can be augmented so that any order statistic can be determined

in $O(\lg n)$ time.

→ Rank of an element in an ordered set is the position of the element in the linear order of the set.

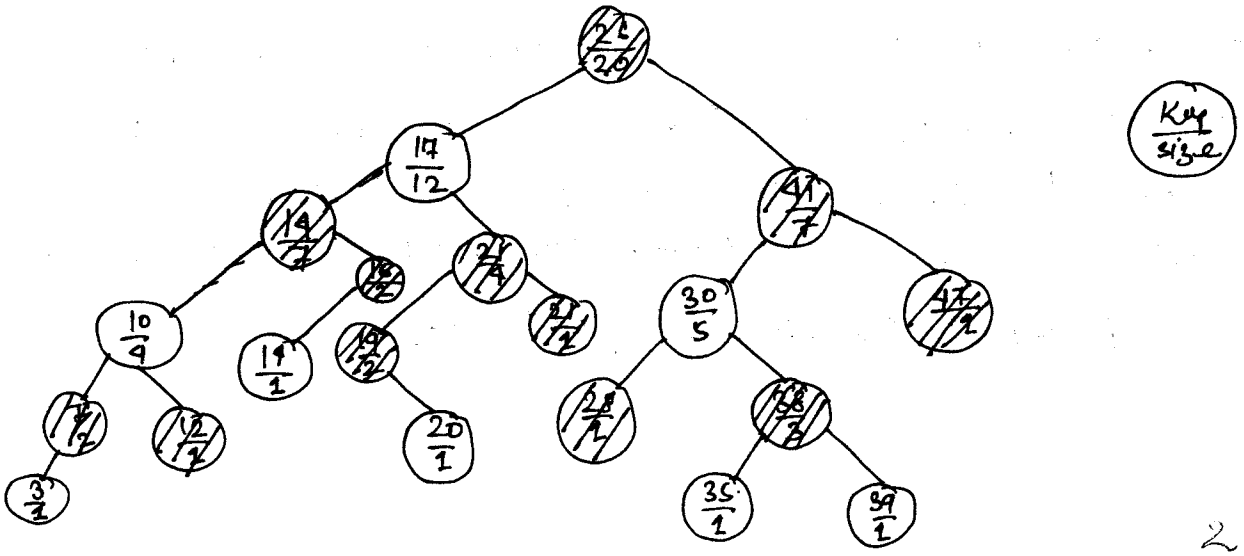
3) Order-Statistic tree

An order-statistic tree is a red-black tree with size information stored in each node.

For any node x, size[x] is the size of the subtree rooted at x, i.e. the no. of (internal) nodes in the subtree rooted at x (including x). The normal fields left[x], right[x], parent[x], Key[x] and color[x] are still present.

We define the sentinel's size to be 0. i.e. size[nil[T]] = 0. We then have:-

size[x] = size[left[x]] + size[right[x]] + 1



→ In an order-statistic tree, we do not require the keys to be distinct. For example the above shown tree has 2 keys with value 14.

In the presence of equal keys the above notion of rank is not well defined. Hence, for an order-statistic tree, we define the rank of an element as the position at which it would be printed in an inorder walk of the tree. For example, in the above figure the key 14 stored in a black node has rank 5 while the key 14 stored in a red node has rank 6.

3) Retrieving an element with a given rank from an ordered-statistic tree

The procedure OS_SELECT(x, i) returns a pointer to the node containing the ith smallest key in the subtree rooted at x. To find the ith smallest key in the subtree rooted at T (i.e. the entire order-statistic tree) invoke OS_SELECT(root[T], i).

OS-SELECT(x, i)

```
1    rank ← size[left[x]] + 1
2    if i = rank
3       then return x
4    elseif i < rank
5          then return OS-SELECT(left[x], i)
6    else return OS-SELECT(right[x], i - rank)
```

→ rank of x = size[left[x]] + 1 = the no. of nodes that come before x in an inorder tree walk of the subtree rooted at x + 1

if the input-rank is equal to the rank of x, x is the node containing the ith smallest key. So we return x in that case. If the input-rank is less than the rank of x the ith smallest element is in the left subtree of x. Hence, we invoke OS-SELECT(left[x], i).

if the input-rank > rank of x then the ith smallest element is in the right subtree of x. When we consider the subtree rooted at the right child of x, we would no longer be considering the elements in the left subtree of x or even x itself. Hence, the approach is to find the (i - rank)th smallest element in the subtree rooted at the right child of x.

→ Because each recursive call goes down one level in the order statistic tree, the total time for OS-SELECT is at worst proportional to the height of the tree. Since an order-statistic tree is a red-black tree its height is $O(\lg n)$, where n is the no. of nodes. Thus, the run time of OS-SELECT is $O(\lg n)$ for a dynamic set of n elements.

4) Determining the rank of an element in an ordered-static tree

Given a pointer to a node x in an order-statistic tree T, the procedure OS-RANK (T, x) returns the position of x in the linear order determined by an inorder tree walk of T.

OS-RANK(T, x)

```
1   rank ← size[left[x]] + 1
2   trav ← x
3   while trav ≠ root[T]
4         do if trav = right[parent[trav]]
5               then rank ← rank + size[left[
                      parent[trav]]] + 1
6               trav ← parent[trav]
7   return rank
```

→ OS_RANK maintains the following loop invariant: At the start of each iteration of the while loop, rank is the rank of Key[x] at the subtree rooted at trav.

Initialization:

Prior to the 1st iteration rank is set to be the rank of Key[x] in the subtree rooted at x. trav is set to x. Thus, the loop invariant is true prior to the 1st iteration of the loop.

Maintenance:

If trav is a left child of parent of trav, then neither parent[trav] nor any key in the subtree rooted at right[parent[trav]] precedes x, so we leave rank alone. Otherwise, trav is a right child and all the nodes in parent[trav]'s left subtree precede as, as does parent[trav] itself. Thus, we update ~~trav~~ rank by adding size[left[parent[trav]]] + 1.

Next, we move one level up the tree.

Termination:

When trav = root[T], the subtree rooted at trav is the entire tree. Thus, the value of rank is the rank of Key[x] in the entire tree.

→ Since x goes up one level in the tree with each iteration, and each iteration of the while loop takes O(1) time, the running time of OS-RANK is at worst proportional to the height of the tree: O(lg n) for an n-node order-statistic tree.

5) Maintaining subtree sizes

Subtree sizes can be maintained for both insertion and deletion without affecting the asymptotic running time of either operation.

(i) Size updation during insertion:

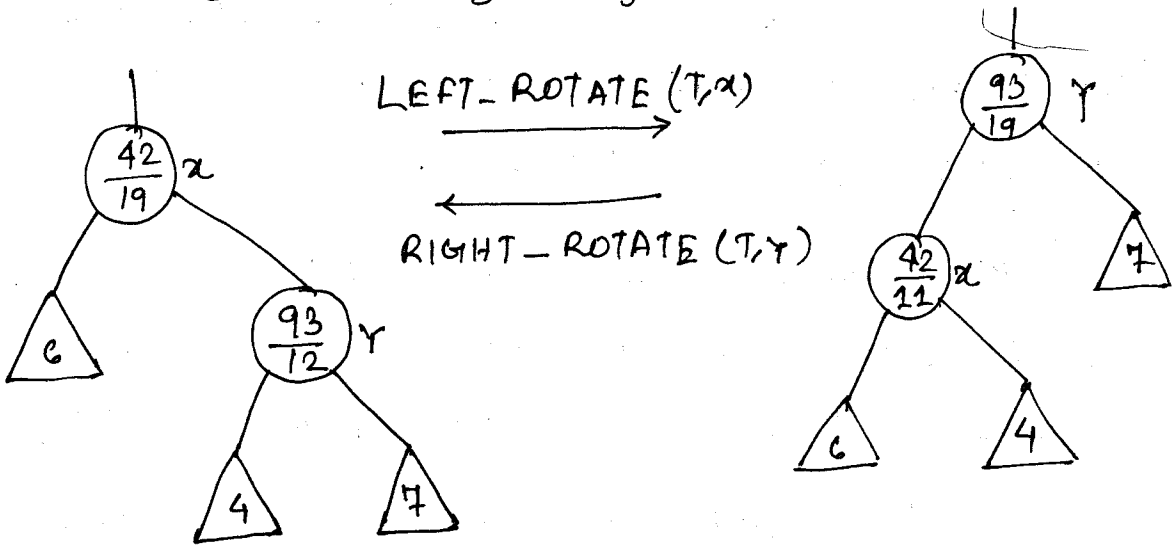
Insertion into a red-black tree comprises of 2 phases:

(1) the first phase goes down the tree from the root, inserting the new node as a child of an existing node. To maintain subtree sizes in the first phase, we simply increment size[x] for each node x on the path traversed from the root down toward the leaves. The newly added node gets a size of 1. Since there are O(lg n) nodes on the traversed path, the additional cost of maintaining the size fields is O(lg n).

(2) in the second phase, the only structural changes to the underlying red-black tree are caused by rotations, of which there are at most two. A rotation is a local operation: only 2 nodes have their size fields invalidated. We can add the following lines at the end of LEFT_ROTATE(T,x)

12 size[y] ← size[x]

13 size[x] ← size[left[x]] + size[right[x]] + 1



Since at most 2 rotations are performed during insertion into a red-black tree, only O(1) additional time is spent updating size fields in the second phase.

Thus, the total time for insertion into an n-node order-statistic tree is O(lg n), which is asymptotically the same as for a red-black tree.

(ii) Size updation during deletion:

Deletion from a red-black tree also comprises of two phases

(1) the first phase splices out the node spliced-node. To update subtree sizes we simply traverse a path from spliced-node up to the root, decrementing the size field of each node on the path. Since, this path is $O(\lg n)$ in an n-node red-black tree, the additional time spent maintaining the size fields in the first phase is $O(\lg n)$.

(2) the second phase causes at most 3 rotations and otherwise performs no structural changes. Thus the second phase is $O(1)$.

Thus, both insertion and deletion, including the maintainence of the size fields take $O(\lg n)$ time for an n-node order-statistic tree.

Continuation from 26 pages after →